

Revealing Inherent Concurrency in Event Data: A Partial Order Approach to Process Discovery

Humam Kourani^{1,2}[0000–0003–2375–2152], Gyunam Park¹[0000–0001–9394–6513],
and Wil M.P. van der Aalst^{1,2}[0000–0002–0955–6940]

¹ Fraunhofer FIT, Schloss Birlinghoven, 53757 Sankt Augustin, Germany
{humam.kourani,gyunam.park,wil.van.der.aalst}@fit.fraunhofer.de

² RWTH Aachen University, Ahornstraße 55, 52074 Aachen, Germany

Abstract. Process discovery algorithms traditionally linearize events, failing to capture the inherent concurrency of real-world processes. While some techniques can handle partially ordered data, they often struggle with scalability on large event logs. We introduce a novel, scalable algorithm that directly leverages partial orders in process discovery. Our approach derives partially ordered traces from event data and aggregates them into a sound-by-construction, perfectly fitting process model. Our hierarchical algorithm preserves inherent concurrency while systematically abstracting exclusive choices and loop patterns, enhancing model compactness and precision. We have implemented our technique and demonstrated its applicability on complex real-life event logs. Our work contributes a scalable solution for a more faithful representation of process behavior, especially when concurrency is prevalent in event data.

Keywords: process discovery, partial order, concurrency, interval event data

1 Introduction

Process discovery, a fundamental pillar of process mining, focuses on automatically generating a process model from such an event log, providing a visual and analytical representation of the underlying process behavior.

Traditional process discovery techniques assume totally ordered event data, where each process instance is a strict activity sequence (e.g., $\langle a, b, c \rangle$). This clashes with the reality of complex processes where activities frequently execute concurrently or overlap in time. While information systems may record interval data (e.g., start and completion timestamps), conventional methods often discard this by selecting a single timestamp, losing information about true execution semantics. Even with atomic events, events might share the exact same timestamp, naturally highlighting their lack of strict sequential dependency. Furthermore, recorded timestamps may be unreliable, suffer from varying granularities, or be recorded at a level too fine-grained to reveal meaningful process structure [1]. Discrepancies between actual event occurrence and recording times are common, particularly in domains like healthcare where manual data entry delays are frequent [15]. In such cases, an abstraction of timestamps to a coarser level (e.g., hour, day, or even coarser business-defined periods) becomes necessary.

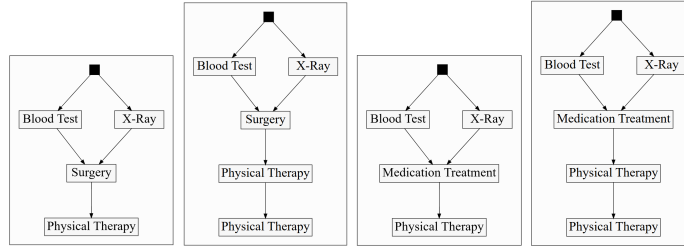


Fig. 1: Trace variants for the hospital process, represented as four partial orders.

Consider a simplified hospital patient journey. A nurse takes a blood sample but logs it hours later (e.g., at the end of the shift). Later that evening, the patient undergoes an X-ray, but only the date of this procedure is logged (00:00 timestamp). In the next days, a choice is made: either a surgery is needed or a medication treatment is sufficient. Towards the end of the process, one or possibly two courses of physical therapy are prescribed. In this hospital scenario, imposing a total order based on raw timestamps would be misleading. Abstracting timestamps to the ‘day’ level is necessary, revealing underlying partial orders, as shown by the four trace variants in Fig. 1. This example underscores the necessity for discovery techniques for partially ordered data. Traditional approaches that assume an arbitrary order for events with identical timestamps introduce fictitious sequential dependencies that misrepresent the actual process logic.

This paper proposes a novel process discovery approach that leverages the Partially Ordered Workflow Language (POWL) [9]. POWL is a hierarchical process modeling language where submodels are combined to form larger ones, either as partial orders or using control-flow operators: $\times$ for exclusive choice and $\circ$ for repeated behavior. Crucially, POWL guarantees soundness by construction, meaning that POWL models can be translated into Workflow nets that inherently free from common structural issues such as deadlocks, a highly desirable trait for process discovery. Furthermore, POWL’s design, which inherently supports partial orders as a fundamental modeling construct, makes it an ideal process modeling language for discovering processes from partially ordered data. Our methodology first derives partial orders from event data. Our algorithm then merges them into a general, encompassing partial order. It also incorporates additional steps to detect common control-flow patterns: mutually exclusive activities are grouped using $\times$, and repeatable blocks with $\circ$. This results in a POWL model that can more accurately capture real-life process behaviors, especially their inherent concurrency.

2 Related Work

While POWL has been used for process discovery in [8, 10], these existing techniques follow traditional discovery paradigms that operate on the assumption of totally ordered traces. The investigation of process mining techniques that handle inherent concurrency in event data has become a significant and growing research area, as surveyed in [13]. In [12], the authors introduce an

Inductive Miner variant that leverages lifecycle transition data to differentiate concurrency from sequential interleaving. Similarly, the Split Miner is refined in [2] to detect overlapping execution intervals when start/end timestamps are available. In [6], the authors advocate for using prime event structures as a unified representation for both process models and event logs, highlighting their natural ability to capture concurrency. The Prime Miner [3] leverages lifecycle data to discover prime event structures from event logs and finally synthesize a Petri net via region-based techniques.

The Multi-Phase Miner [17] aggregates instance graphs into higher-level models. Further exploring aggregation, the authors in [5] detail algorithms for constructing workflow nets from sets of partially-ordered causal runs. The eST² Miner [16] combines a replay-based discovery algorithm with Petri net synthesis from partial orders. The ILP² Miner [7] uses Integer Linear Programming for discovery from partially ordered event logs. The Zebra Miner [11] employs an incremental region-based technique to synthesize Petri nets from partial orders. Another synthesis approach uses Petri net unfoldings, where an expert-provided independence relation is used to construct partial orders [14]. Finally, the authors in [4] explore the synthesis of Petri nets by folding partially ordered runs.

Existing techniques, particularly those relying on computationally intensive synthesis like region-based mining or integer linear programming, often face scalability challenges on large, complex event logs [13]. Our hierarchical and aggregative method is designed for efficiency, making it applicable to real-world datasets.

3 Preliminaries

A *multiset* over a set X , denoted $M \in \mathcal{B}(X)$, allows for multiple occurrences of elements from X . For a set X , a *partition* of X of size $n \geq 1$ is a set of subsets $P = \{X_1, \dots, X_n\}$ such that $X = X_1 \cup \dots \cup X_n$, $X_i \neq \emptyset$ for all $1 \leq i \leq n$, and $X_i \cap X_j = \emptyset$ for all $1 \leq i < j \leq n$.

A binary relation $\prec \subseteq X \times X$ over a set X indicates relationships between pairs of elements. We write $x_1 \prec x_2$ for $(x_1, x_2) \in \prec$. The *transitive closure* of $\prec$ is defined as $\prec^+ = \{(x, y) \mid \exists_{x_1, \dots, x_n \in X} x = x_1 \wedge y = x_n \wedge \forall_{1 \leq i < n} x_i \prec x_{i+1}\}$.

A *strict partial order* (or simply *partial order*) $\prec$ over a set X is an irreflexive ($x \not\prec x$ for all $x \in X$) and transitive ($x_1 \prec x_2 \wedge x_2 \prec x_3 \Rightarrow x_1 \prec x_3$) binary relation. These properties imply asymmetry ($x_1 \prec x_2 \Rightarrow x_2 \not\prec x_1$). Given a set X , we use $\mathcal{O}(X)$ to denote the set of all partial orders over X .

Let Σ be the universe of activity labels. An *Event Log* is a collection of events. An event e has at least the following attributes: a case identifier, an activity label $l(e) \in \Sigma$, and a timestamp. Optionally, an event may have a lifecycle attribute to distinguish different phases of the activity execution (e.g., **start**, **complete**, **suspend**, and **resume**). Events that share the same case identifier represent the execution of a single process instance (often referred to as a *trace* or *case*).

We use $\tau \notin \Sigma$ to denote a silent activity, which represents an unobservable action. Let $\mathcal{T}$ be the universe of transitions. Each transition $t \in \mathcal{T}$ has an associated activity label $l(t) \in \Sigma \cup \{\tau\}$. Transitions having the same label represent

multiple instances of an activity. In this paper, we conceptualize a transition t as a pair (a, k) where $a = l(t)$ is its label and $k = idx(t)$ is an index that distinguishes it from other transitions with the same label within the same process instance. For example, we use $t_1 = (a, 1) \in \mathcal{T}$ and $t_2 = (a, 2) \in \mathcal{T}$ to represent two distinct instances of an activity $a \in \Sigma$.

The Partially Ordered Workflow Language (POWL) [9] is a hierarchical process modeling language. POWL models can be combined to form larger ones using a partial order, the choice operator ($\times$), or the loop operator ($\odot$).

Definition 1 (POWL Model). *POWL models are defined recursively:*

- Any transition $t \in \mathcal{T}$ is a POWL model.
- Let $P = \{\psi_1, \dots, \psi_n\}$ be a set of $n \geq 2$ POWL models. Then:
 - $\times(P)$ is a POWL model.
 - $\odot(\psi_1, \psi_2)$ is a POWL model.
 - A partial order $\prec \in \mathcal{O}(P)$ is a POWL model.

The semantics of a POWL model are given by the set of activity sequences it can produce. For $\times$, only one of the submodels is executed. For $\odot$, the first submodel (do-part) is executed, then zero or more repetitions of the second submodel (redo-part) followed by the do-part are executed. In a partial order, all submodels are executed while respecting the given execution order.

Two POWL models are considered equivalent if they are of the same type and their children are recursively equivalent. For atomic transitions, equivalence is based on having the same activity label.

Definition 2 (Equivalent POWL Models). *Let ψ and ψ' be two POWL models. The two models are equivalent, denoted as $\psi \sim \psi'$, if one of the following conditions is met:*

- $\psi \in \mathcal{T} \wedge \psi' \in \mathcal{T} \wedge l(\psi) = l(\psi')$.
- Two sets of $n \geq 2$ POWL models $P = \{\psi_1, \dots, \psi_n\}$ and $P' = \{\psi'_1, \dots, \psi'_n\}$ exist such that $\psi_i \sim \psi'_i$ for any $1 \leq i \leq n$ and one of the following conditions is met:
 - $\psi = \times(P)$ and $\psi' = \times(P')$.
 - $n = 2$, $\psi = \odot(\psi_1, \psi_2)$, and $\psi' = \odot(\psi'_1, \psi'_2)$.
 - $\psi = \prec \in \mathcal{O}(P)$ and $\psi' = \prec' \in \mathcal{O}(P')$ such that $\psi_i \prec \psi_j \Leftrightarrow \psi'_i \prec' \psi'_j$ for all $i, j \in \{1, \dots, n\}$.

We use $\mathcal{L}(\psi)$ to denote the set of all activity labels present within the hierarchical structure of a POWL model ψ . For example, if $\psi = \odot(\times((a, 1), (b, 1)), (a, 2))$, then $\mathcal{L}(\psi) = \{a, b\}$.

4 Deriving Partial Orders from Event Data

To enable our process discovery approach, raw event data must first be transformed. This section details the critical preparatory stage: the systematic conversion of a standard event log into a collection of partial orders.

An *interval event log* L_{int} is a collection of *interval events*, where each interval event e represents a single, complete execution of an activity. Formally, an interval event can be represented as a tuple $e = (a, c, st, et)$, where $a \in \Sigma$ is the activity label, c is the case identifier, st is the start timestamp, and et is the completion timestamp, with $st \leq et$. We use $\mathcal{C}(L_{int})$ to denote the set of all case identifiers in an interval event log, i.e., $\mathcal{C}(L_{int}) = \{c \mid (a, c, st, et) \in L_{int}\}$.

The precise method of deriving an interval event log L_{int} from a raw event log L can vary and is highly configurable. In our work, we employ a method that matches ‘start’ and ‘complete’ lifecycle transitions for the same activity label within each case on a First-In, First-Out (FIFO) basis to form interval events. If lifecycle information is absent or any ‘complete’ events could not be matched via the FIFO process, then atomic interval events are created (i.e., with identical start and end timestamps). This approach, similarly to the one in [12], enables the direct application of the downstream discovery algorithm to any event log, even when lifecycle information is incomplete.

Once L_{int} is obtained, we can construct a partial order to represent each process instance.

Definition 3 (Partially Ordered Trace). *Let L_{int} be an interval event log and $c \in \mathcal{C}(L_{int})$ be a case identifier in L_{int} . Let $e_1 = (a_1, c, st_1, et_1), \dots, e_n = (a_n, c, st_n, et_n) \in L_{int}$ be all interval events that have c as their case identifier in L_{int} . The corresponding Partially Ordered Trace (POT) P_c is a pair $(V_c, \prec_c)$, where:*

- $V_c = \{t_1, \dots, t_n\}$ the a set of transitions, where each transition corresponds to an interval event, i.e., $l(t_i) = a_i$ for all $1 \leq i \leq n$.
- $\prec_c \subseteq V_c \times V_c$ is a binary relation defined as follows for any $i, j \in \{1, \dots, n\}$: $t_i \prec_c t_j \Leftrightarrow et_i < st_j$.

For consistency, we always use positive integers (1, 2, ...) as indexes for transitions sharing the same activity label within a POT, ordered by their start and end timestamps. For example, if two interval events within a case c share the same label a , the one with the earlier start timestamp is mapped into $(a, 1)$, and the one starting later is mapped into $(a, 2)$.

The relation $\prec_c$ in Def. 3 captures the temporal precedence between activity instances: $t_i \prec_c t_j$ if e_i completes strictly before e_j starts. This definition naturally handles concurrency, as activity instances whose execution intervals overlap will not have a precedence relation between them. It is straightforward to show that $\prec_c$ is a strict partial order. The use of transitions ensures that even if multiple instances share the same activity label, they are treated as distinct nodes in V_c , allowing $\prec_c$ to be well-defined as a strict partial order based on the individual start and completion times.

Given an interval event log L_{int} , the multiset of all POTs derived from L_{int} serves as the direct input to our POWL discovery algorithm.

Definition 4 (Partially Ordered Event Log). *Let L_{int} be an interval event log. The partially ordered event log corresponding to L_{int} is the multiset of POTs defined as follows: $L_{PO} = [P_c \mid c \in \mathcal{C}(L_{int})]$.*

5 POWL Discovery from Partial Orders

This section presents our algorithm for discovering a POWL process model from a multiset of POTs. The core of our algorithm is the aggregation of the input partial orders into a single, encompassing partial order that captures the overall process structure. The algorithm also incorporates additional steps that aim to identify and abstract exclusive choice, repeated, and co-occurring patterns.

5.1 Notation and Basic Operations

Let M be a multiset of partial orders. The set of all unique nodes present across all partial orders in M is denoted by $\mathcal{V}(M) = \bigcup_{(V, \prec) \in M} V$.

During recursive discovery, it's often necessary to isolate the behavior related to a specific subset of nodes. For instance, when an XOR is detected, we need to independently discover a model for each of its branches. The *partial order projection* operation enables this by creating a new multiset of partial orders that only contain elements from a specified subset of nodes.

Definition 5 (Partial Order Projection). *Let M be a multiset of partial orders and let $V_{sub} \subseteq \mathcal{V}(M)$ be a subset of nodes in M . Then the projection of M on V_{sub} is defined as follows:*

$$M|_{V_{sub}} = [(V', \prec \cap (V' \times V')) \mid (V, \prec) \in M \\ \wedge V' = V \cap V_{sub} \wedge V' \neq \emptyset].$$

Once a structural pattern (e.g., an XOR) involving a set of nodes V_{sub} is identified and a corresponding POWL submodel ψ is created, the algorithm replaces V_{sub} with ψ in the current multiset of partial orders M . The node substitution must ensure that the new node ψ correctly inherits the precedence relations of the set V_{sub} it replaces.

Definition 6 (Node Substitution in Partial Orders). *Let M be a multiset of partial orders and let $V_{sub} \subseteq \mathcal{V}(M)$ be a subset of nodes in M . Let $\psi \notin \mathcal{V}(M)$ be a new node not present in M . The substitution of V_{sub} by ψ in M is defined as*

$$M_{V_{sub} \rightarrow \psi} = [(V', \prec') \mid (V, \prec) \in M]$$

where

$$V' = \begin{cases} V & \text{if } V \cap V_{sub} = \emptyset, \\ (V \setminus V_{sub}) \cup \{\psi\} & \text{otherwise.} \end{cases}$$

and

$$\prec' = \{(s, t) \mid s \prec t \wedge s \notin V_{sub} \wedge t \notin V_{sub}\} \\ \cup \{(\psi, t) \mid t \in V \wedge V \cap V_{sub} \neq \emptyset \wedge \forall s \in V \cap V_{sub} s \prec t\} \\ \cup \{(s, \psi) \mid s \in V \wedge V \cap V_{sub} \neq \emptyset \wedge \forall t \in V \cap V_{sub} s \prec t\}.$$

5.2 Mining for Exclusive Choices

Our discovery algorithm identifies candidates for exclusive choice (XOR) constructs by analyzing the co-occurrence patterns of activities in the input POTs.

If certain activities (or sets of activities) are never observed to occur together within the same process instance (trace), they are likely to be mutually exclusive alternatives. To formalize this, we first define a conflict relation between individual activity labels based on their absence of co-occurrence.

Definition 7 (Activity Conflict Relation). *Let M be a multiset of partial orders, and let a and b be two activity labels present in M , i.e., $a, b \in \bigcup_{\psi \in \mathcal{V}(M)} \mathcal{L}(\psi)$. We say that a and b are conflicting in M , denoted as $a \#_M b$, if and only if:*

$$\nexists_{(V, \prec) \in M; \psi, \psi' \in V} a \in \mathcal{L}(\psi) \wedge b \in \mathcal{L}(\psi').$$

An *activity conflict group* extends this concept to a partitioning of a set of activities, where each part (a set of activity labels) is mutually exclusive with every other part in the partition.

Definition 8 (Activity Conflict Group). *Let M be a multiset of partial orders. Let $A \subseteq \bigcup_{\psi \in \mathcal{V}(M)} \mathcal{L}(\psi)$ be a set of activity labels present in M . Let $G = \{A_1, \dots, A_n\}$ be a partitioning of A into $n \geq 2$ parts. We say that G is a conflict group in M if and only if for all $A_i, A_j \in G$:*

$$A_i \neq A_j \Rightarrow \forall_{a_i \in A_i, a_j \in A_j} a_i \#_M a_j.$$

We use $\#(M)$ to denote the set of all conflict groups in M .

Maximal conflict groups are those conflict groups not contained within any larger ones, i.e., the set of maximal conflict groups in M is:

$$\#^{max}(M) = \{G \in \#(M) \mid \nexists_{G' \in \#(M)} G \subset G'\}.$$

5.3 Mining for Loops

To discover loop constructs, our algorithm needs to identify instances of submodels that are semantically identical within the partial orders. Based on the equivalence relation (cf. Def. 2), the set of all distinct POWL models in a multiset of partial orders can be partitioned into *POWL equivalent classes*. Each class groups together all models that are semantically identical.

Definition 9 (POWL Equivalence Classes). *Let V be a set of POWL models. The equivalence classes of V is a partitioning of V defined as follows:*

$$\sim(V) = \{\{\psi' \in V \mid \psi' \sim \psi\} \mid \psi \in V\}.$$

If an equivalence class in $\sim(\mathcal{V}(M))$ contains more than one model, it signifies that this particular model structure is repeated within M . All instances within such an equivalence class are then abstracted into a single Loop model, where one instance forms the do-part of the loop.

5.4 Identifying Coupled Components

In many processes, certain sets of activities or sub-processes exhibit strong synchronous behavior: they tend to appear together or are entirely absent together across all observed instances. Such groups represent logically cohesive

blocks of work. Identifying and abstracting these *co-occurring groups* significantly enhances the accuracy of other pattern detection steps (e.g., it enables the discovery of loops at a higher level of abstraction).

The *node co-occurrence relation* captures whether two nodes always appear together within the same partial order.

Definition 10 (Node Co-Occurrence Relation). *Let M be a multiset of partial orders. Let $\psi, \psi' \in \mathcal{V}(M)$ be two nodes in M . We say that ψ and ψ' co-occur in M , denoted by $\psi \leftrightarrow_M \psi'$, if and only if:*

$$\forall (V, \prec) \in M \quad \psi \in V \Leftrightarrow \psi' \in V.$$

We use the node co-occurrence relation to partition the set of all nodes into disjoint subsets, where all nodes within a particular subset always co-occur.

Definition 11 (Node Co-Occurrence Partitioning). *Let M be a multiset of partial orders. The co-occurrence partitioning of the nodes of M is defined as follows:*

$$\leftrightarrow(M) = \{\{\psi' \in \mathcal{V}(M) \mid \psi' \leftrightarrow_M \psi\} \mid \psi \in \mathcal{V}(M)\}.$$

5.5 Combining Partial Orders

After abstracting control-flow patterns, the remaining nodes (which can be atomic transitions or more complex POWL submodels) are aggregated into a single, encompassing partial order that represents the overall process behavior.

Definition 12 (Combining Partial Orders). *Let M be a multiset of partial orders. The function **CombineOrders**(M) returns a single aggregated partial order $(V_{agg}, \prec_{agg})$ where $V_{agg} = \mathcal{V}(M)$ and $\prec_{agg}$ is constructed as follows:*

- **Base Precedence Relation:** *Let $\prec_{base} \subseteq V_{agg} \times V_{agg}$ be the relation defined as follows for any $u, v \in V_{agg}$:*

$$u \prec_{base} v \Leftrightarrow \exists (V, \prec) \in M \quad u \prec v \wedge \nexists (V, \prec) \in M \left(\{u, v\} \subseteq V \wedge u \not\prec v \right).$$

This relation captures sequential dependencies that are present and never contradicted in the input partial orders.

- **Extended Precedence Relation:** *Let $\prec_{ext} \subseteq V_{agg} \times V_{agg}$ be the relation defined as follows for any $u, v \in V_{agg}$:*

$$u \prec_{ext} v \Leftrightarrow u \prec_{base}^+ v \wedge \nexists (V, \prec) \in M \left(\{u, v\} \subseteq V \wedge u \not\prec v \right).$$

This relation cautiously extends $\prec_{base}$ by inferring additional dependencies by transitivity unless they contradict the input partial orders.

- **Transitive Aggregated Relation:** $\prec_{agg} = \text{Prune}(\prec_{ext})$, where **Prune** is function that iteratively identifies and resolves violations of transitivity to ensure $\prec_{agg}$ is a partial order. For instance, if $v_1 \prec_{ext} v_2$ and $v_2 \prec_{ext} v_3$ but $v_1 \not\prec_{ext} v_3$, an edge (i.e., (v_1, v_2) or (v_2, v_3)) is removed. This pruning

Algorithm 1: POWL Discovery from Partial Orders**Input:** A non-empty multiset of partial orders M **Output:** A single POWL model**Function** DiscoverPOWL(M):

```

1   Step 1: XOR Mining
2   for  $G = \{A_1, \dots, A_n\} \in \#^{max}(M)$  do
3       for  $1 \leq i \leq n$  do
4            $V_i \leftarrow \{\psi \in \mathcal{V}(M) \mid \mathcal{L}(\psi) \subseteq A_i\}$ 
5            $\psi_i \leftarrow \text{DiscoverPOWL}(M|_{V_i})$ 
6            $\psi_G \leftarrow \times(\psi_1, \dots, \psi_n)$ 
7            $M \leftarrow M_{V_1 \cup \dots \cup V_n \rightarrow \psi_G}$ 

8   Step 2: Co-Occurrence Grouping
9   if  $|\leftrightarrow(M)| > 1$  then
10      for  $V \in \leftrightarrow(M)$  do
11           $\psi_V \leftarrow \text{DiscoverPOWL}(M|_V)$ 
12           $M \leftarrow M_{V \rightarrow \psi_V}$ 

13  Step 3: Loop Mining
14  for  $V \in \sim(\mathcal{V}(M))$  do
15      if  $|V| > 1$  then
16           $\psi \leftarrow \text{random}(V)$ 
17           $\psi_{\odot} \leftarrow \odot(\psi, (\tau, 1))$ 
18           $M \leftarrow M_{V \rightarrow \psi_{\odot}}$ 

19  Step 4: Skip Mining
20  for  $\psi \in \mathcal{V}(M)$  do
21      if  $\exists_{(V, \prec) \in M} \psi \notin V$  then
22           $\psi_{\times} \leftarrow \times(\psi, (\tau, 1))$ 
23           $M \leftarrow M_{\{\psi\} \rightarrow \psi_{\times}}$ 

24  Step 5: Order Aggregation
25   $\psi = (V, \prec) \leftarrow \text{CombineOrders}(M)$ 
26  return  $\psi$ 

```

process is applied iteratively, until no such patterns remain and transitivity is reached. The choice of which edge to remove and the removal order if multiple violations exist will lead to non-deterministic results.

5.6 The Recursive POWL Discovery Algorithm

With the foundational operations and pattern definitions in place, we now present the recursive algorithm for discovering a POWL model from a multiset of POTs, outlined in Alg. 1. Steps 1-3 identify and abstract common process patterns (choices, co-occurring groups, and loops), aiming for a more structured and compact model. The core steps 4-5 handle optionality and aggregate remaining elements into a single partial order.

1. **XOR Mining:** The algorithm identifies maximal conflict groups $A_1, \dots, A_n$. For each A_i , the corresponding set of nodes V_i is determined, and a submodel ψ_i is recursively discovered from the projection $M|_{V_i}$. All branch models are combined with the operator $\times$.
2. **Co-Occurrence Grouping:** The algorithm builds the co-occurrence partitioning $\leftrightarrow(M)$. For each group V , it recursively discovers a POWL model ψ_V from the projection $M|_V$, and it then replaces V with ψ_V .

3. **Loop Mining:** The algorithm partitions $\mathcal{V}(M)$ into POWL equivalent classes $\sim(\mathcal{V}(M))$. If a class V contains multiple instances, a representative model ψ is chosen, and a loop $\odot(\psi, (\tau, 1))$ is created to replace V .
4. **Skip Mining:** For each node $\psi \in \mathcal{V}(M)$, if it's found to be absent in at least one partial order in M , it's replaced with $\times(\psi, (\tau, 1))$.
5. **Partial Order Aggregation:** The algorithm synthesizes a single partial order from the nodes in M , as detailed in Sec. 5.5.

5.7 Discussion: Soundness and Fitness Guarantees

POWL models are inherently *sound* by construction [10]. Beside soundness, our algorithm provides a *perfect fitness* guarantee. In this context, perfect fitness means that any sequence of activities that is a valid linearization of any POT in the input multiset can be replayed by the discovered POWL model. This guarantee primarily stems from (i) the conservative order aggregation (Step 5), which avoids inferring precedences conflicting with input POTs, and (ii) from handling optionality (Step 4), allowing replication of traces with missing parts. These mechanisms, along with the abstraction steps (Steps 1-3), which are also fitness-preserving, ensure that all observed behaviors in the input POTs are permissible in the final POWL model.

6 Implementation and Evaluation

We have implemented our discovery approach in the Python library ‘powl’³. and developed a web application for demonstration purposes⁴.

To evaluate our approach, we conducted an experiment on two well-known, real-life event logs: BPI Challenge 2012 and BPI Challenge 2017⁵. We created several variants of each log by filtering them to retain only the top 4, 6, 8, and 12 most frequent activities, in addition to using the full, unfiltered logs.

We compare our approach against two state-of-the-art techniques: the Zebra Miner [11] and the eST² Miner [16]⁶. To derive the input partial orders for the eST² Miner, we used the ‘PTrace’ plugin in ProM with the ‘timePO’ option selected. To ensure a fair comparison, we configured the eST² Miner with a fitness threshold of 1.0 to produce perfectly fitting models. For other parameters, we used the same defaults selected in [16].

A timeout of one hour was set for each discovery run. The quality of the discovered models was measured using standard alignment-based *fitness* and *precision*, computed with PM4Py⁷. As standard conformance checking algorithms operate on totally ordered traces, these metrics were computed using only the

³ Install via ‘pip install powl’. A usage example is available at https://github.com/humam-kourani/POWL/blob/main/examples/partial_order_based_discovery.py.

⁴ The demonstrator is accessible at <https://po-aware-powl-miner.streamlit.app/>.

⁵ Both event logs are available at <https://data.4tu.nl/>

⁶ Zebra Miner is accessible at <https://www.fernuni-hagen.de/ilovepetrinets/zebra/>, while eST² Miner is available in ProM (<https://promtools.org/>).

⁷ <https://processintelligence.solutions/pm4py>.

Table 1: Evaluation Results. T/O indicates reaching a timeout of one hour.

Event Log		BPI Challenge 2012					BPI Challenge 2017				
Num. Activities		4	6	8	12	Full (24)	4	6	8	12	Full (26)
Time (sec)	POWL Miner	8	12	13	19	25	17	26	24	33	41
	eST ² Miner	145	799	2683	T/O	T/O	897	T/O	T/O	T/O	T/O
Precision	POWL Miner	0.53	0.51	0.56	0.45	0.21	0.95	0.71	0.56	0.53	0.27
	eST ² Miner	0.45	0.39	0.46	-	-	0.91	-	-	-	-

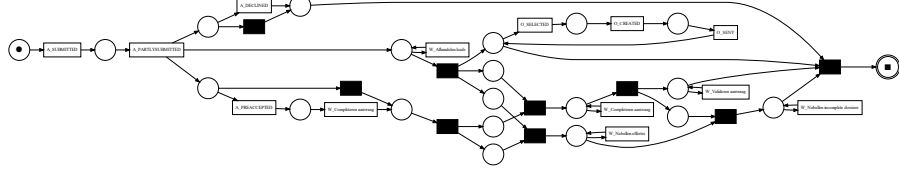


Fig. 2: The POWL model discovered from the BPI Challenge 2012 event log with 12 activities translated into a Petri net.

‘complete’ instances from the event logs. These derived sequential traces represent valid linearizations of the more general partially ordered traces.

Results All models generated during the experiment are available at <https://github.com/humam-kourani/results-po-aware-powl-miner>. The results of our experiment are summarized in Tab. 1. The results for the Zebra Miner are excluded as it reached the time limit on all log variants. We don’t report fitness values as both the POWL Miner and eST² Miner achieved perfect fitness on all logs due to their fitness guarantees.

In terms of precision, our approach consistently outperformed the eST² Miner. This suggests that our methodology produces a more faithful and less over-generalized representation of the process. For example, Fig. 2 shows the model discovered for the BPIC 2012 log with 12 activities. Regarding performance, our algorithm demonstrates high scalability. It successfully processed all log variants, with the longest discovery task completing in just 42 seconds for the full BPIC 2017 log. In contrast, the eST² Miner was significantly slower and encountered the one-hour timeout on the BPIC 2012 log with 12 activities and already on the BPIC 2017 log with just 6 activities, failing to produce a model for the more complex scenarios.

7 Conclusion and Future Directions

This paper presented a novel process discovery algorithm that leverages the Partially Ordered Workflow Language (POWL) to discover process models from partially ordered event data. Recognizing the limitations of traditional techniques that often linearize concurrent behavior, our approach leverages the partial order nature of events by first transforming process traces into partial orders. Building upon this foundation, a hierarchical mining algorithm systematically combines these partial orders while identifying and abstracting choice (XOR) and loop constructs. The resulting models are inherently sound and achieve perfect

fitness with respect to the input partial orders. We demonstrated the scalability of our approach by applying it on complex real-life event data.

An important avenue for future work is to investigate incorporating noise filtering mechanisms to abstract from infrequent or exceptional behaviors. We believe this research contributes to advancing process discovery by providing a more faithful and understandable representation of operational processes where activities frequently overlap or run in parallel.

Acknowledgments. This work was funded by the Federal Ministry of Research, Technology and Space (BMFTR), Germany (grant 01IS23065).

References

1. Arthur H. M. ter Hofstede et al.: Process-data quality: The true frontier of process mining. *ACM J. Data Inf. Qual.* **15**(3), 29:1–29:21 (2023)
2. Augusto, A., Dumas, M., Rosa, M.L.: Automated discovery of process models with true concurrency and inclusive choices. In: *ICPM 2020 Workshops*. pp. 43–56 (2020)
3. Bergenthum, R.: Prime miner - process discovery using prime event structures. In: *ICPM 2019*. pp. 41–48 (2019)
4. Bergenthum, R., Mauser, S.: Folding partially ordered runs. In: *Applications of Region Theory 2011*. pp. 52–62 (2011)
5. van Dongen, B.F., Desel, J., van der Aalst, W.M.P.: Aggregating causal runs into workflow nets. *Trans. Petri Nets Other Model. Concurr.* **6**, 334–363 (2012)
6. Dumas, M., García-Bañuelos, L.: Process mining reloaded: Event structures as a unified representation of process models and event logs. In: *PETRI NETS 2015*. pp. 33–48 (2015)
7. Folz-Weinstein, S., Bergenthum, R., Desel, J., Kovár, J.: ILP² miner - process discovery for partially ordered event logs using integer linear programming. In: *PETRI NETS 2023*. pp. 59–76 (2023)
8. Kourani, H., Schuster, D., van der Aalst, W.M.P.: Scalable discovery of partially ordered workflow models with formal guarantees. In: *ICPM 2023*. pp. 89–96 (2023)
9. Kourani, H., van Zelst, S.J.: POWL: partially ordered workflow language. In: *BPM 2023*. pp. 92–108 (2023)
10. Kourani, H., van Zelst, S.J., Schuster, D., van der Aalst, W.M.P.: Discovering partially ordered workflow models. *Inf. Syst.* **128**, 102493 (2025)
11. Kovár, J., Bergenthum, R.: Exploratory process discovery for petri nets. In: *AWPN 2024* (2024)
12. Leemans, S.J.J., Fahland, D., van der Aalst, W.M.P.: Using life cycle information in process discovery. In: *BPM 2015 Workshops*. pp. 204–217 (2015)
13. Leemans, S.J.J., van Zelst, S.J., Lu, X.: Partial-order-based process mining: a survey and outlook. *Knowl. Inf. Syst.* **65**(1), 1–29 (2023)
14. de León, H.P., Rodríguez, C., Carmona, J., Heljanko, K., Haar, S.: Unfolding-based process discovery. In: *ATVA 2015*. pp. 31–47 (2015)
15. Martin, N.: Data quality in process mining. In: Fernandez-Llatas, C. (ed.) *Interactive Process Mining in Healthcare*. pp. 53–79. Springer (2020)
16. Sabine Folz-Weinstein et al.: eST² miner-process discovery based on firing partial orders. *arXiv preprint arXiv:2504.08372* (2025)
17. Van Dongen, B.F., Van der Aalst, W.M.: Multi-phase process mining: Aggregating instance graphs into EPCs and Petri nets. In: *PNCWB 2005*. pp. 35–58 (2005)